Preston-Check — Operator Runbook

Deployed URLs, resource IDs, deploy procedures, emergency response

Preston-Check Maintainers

2026-05-04

Operator Runbook

The single-page reference for everything deployed. Bookmark this file and the page-rendered version at `https://admin.preston-check.com/` (once admin portal includes it).

This file is intentionally in the public repo because it contains zero credentials — only public URLs, non-sensitive resource identifiers, and procedures. The actual secrets (Cloudflare API token, license signing private key) live only in repo secrets and the operator's hardware-bound machine.

Production URLs

SURFACE	URL	HOSTED ON	AUTH
Public landing	`https://preston-check.com/`	GitHub Pages (custom domain via Route 53 A records)	none
GitHub Pages mirror	`https://preston-check.github.io/preston-check/`	GitHub Pages (default subdomain)	none
Customer portal	`https://app.preston-check.com/`	Cloudflare Pages (`preston-check-customer` project)	none yet (production will add magic-link)
Admin portal	`https://admin.preston-check.com/`	Cloudflare Pages (`preston-check-admin` project)	Cloudflare Access — operator email + 6-digit PIN
Telemetry endpoint	`https://app.preston-check.com/api/v1/telemetry`	Cloudflare Pages Function on	none (public POST endpoint by design)

SURFACE	URL	HOSTED ON	AUTH
		customer-portal project	
Standalone telemetry Worker	`https://preston-check-telemetry.preston-check-edge.workers.dev/`	Cloudflare Workers	none (alternative endpoint, same D1+KV)

`*.pages.dev` mirrors (also serve, but use the custom domains in production): - `https://preston-check-admin.pages.dev/` - `https://preston-check-customer.pages.dev/`

Cloudflare resources

RESOURCE	TYPE	ID / NAME	NOTES
Pages project	Customer portal	`preston-check-customer`	Auto-deploys from `web/custome` on each master push
Pages project	Admin portal	`preston-check-admin`	Auto-deploys from `web/admin/`
Workers script	Telemetry collector	`preston-check-telemetry`	Standalone Worker; deploys from `workers/telemetry/`
D1 database	Telemetry storage	`preston-check-telemetry` (`e206e1e4-1c78-4a5e-a983-bc47104d1b3c`)	ENAM region. Schema in `workers/telemetry/schema.sc`
KV namespace	Aggregations + rate-limit	`AGGREGATE` (`330983ec5b464dab8ae2f338d40512f5`)	Used by both the Worker and the Pages Function
Workers subdomain	Account	`preston-check-edge.workers.dev`	Renamed from default to be anonymity-clean
Access app	Operator gate	“Preston-Check Admin” → `admin.preston-check.com`	One-time PIN to operator email; 24 sessions
Pages bindings	DB + AGGREGATE + env	On customer-portal project (production env)	Set via Cloudflare API; visible in dashboard → Pages → preston-check-customer → Settings → Functions

DNS records

In Route 53 hosted zone `preston-check.com.`:

RECORD	TYPE	VALUE	TTL	PURPOSE
`preston-check.com.`	A (×4)	`185.199.108-111.153`	default	GitHub Pages apex
`admin.preston-check.com.`	CNAME	`preston-check-admin.pages.dev`	300	Admin portal
`app.preston-check.com.`	CNAME	`preston-check-customer.pages.dev`	300	Customer portal

(Hosted zone ID `Z05163633TTJD7W45GALS`. The domain itself is registered at Route 53, separate from the hosted zone.)

GitHub repository

	VALUE
Org / repo	`preston-check/preston-check`
Default branch	`master`
Visibility	Public (since 2026-05-04)
Latest tag	`v1.7.5`
License	Apache 2.0
Homebrew tap	`preston-check/homebrew-tap` (separate repo)
GitHub Action	`preston-check/scan-action` (separate repo)

Repository secrets

Set under repo Settings → Secrets and variables → Actions. Never logged, never echoed in workflow output.

SECRET	USED BY	NOTES
`CLOUDFLARE_API_TOKEN`	admin-pages, customer-pages, telemetry-deploy	Scoped to Cloudflare Pages + Workers + KV + D1; rotate via

SECRET	USED BY	NOTES
		dashboard → My Profile → API Tokens → Roll
`CLOUDFLARE_ACCOUNT_ID`	same workflows	Not a credential — Cloudflare account identifier
`HOMEBREW_TAP_TOKEN`	release.yml (homebrew job)	Optional — if absent, formula is bumped manually. PAT with write access to `preston-check/homebrew-tap`
`DOCKERHUB_USERNAME`	release.yml (docker job)	Optional — if absent, Docker image is not pushed
`DOCKERHUB_TOKEN`	release.yml (docker job)	Optional

GitHub Actions workflows

WORKFLOW	TRIGGER	PURPOSE
`pages.yml`	Push to master touching `web/landing/**`	Deploys public landing to GitHub Pages
`admin-pages.yml`	Push touching `web/admin/**` or shared assets	Deploys admin portal to Cloudflare Pages
`customer-pages.yml`	Push touching `web/customer/**` or shared assets	Deploys customer portal + Pages Function (telemetry endpoint)
`telemetry-deploy.yml`	Push touching `workers/telemetry/**`	Deploys standalone Worker (alternative to Pages Function)
`release.yml`	Tag push (`v*`)	Build tarball + sha256 + GitHub Release; Docker image; Homebrew bump
`test.yml`	Every push + PR	Shell syntax check + smoke scans + framework filter test
`lint-community.yml`	PR touching `checks/community/**`	Shellcheck + lint-check.sh on community contributions

WORKFLOW	TRIGGER	PURPOSE
`threat-intel-sync.yml`	Mondays 09:00 UTC + manual dispatch	Pulls fintech-relevant CVEs from NIST NVD; opens PR with draft checks

Health checks

A trivial bash one-liner that confirms every public surface is up:

```
for u in \
  https://preston-check.com/ \
  https://app.preston-check.com/ \
  https://admin.preston-check.com/ \
  https://app.preston-check.com/api/v1/telemetry; do
  printf "%-55s " "$u"
  curl -so /dev/null -w "%{http_code}\n" "$u" --max-time 6
done
```

Expected codes: `200 200 302 405` (admin → 302 because Access redirect; telemetry → 405 because GET without POST is method-not-allowed).

Deploy procedures

Standard release flow

```
# 1. Bump PRESTON_VERSION in preston-check.sh
# 2. Update CHANGELOG.md
# 3. Commit
git commit -m "vX.Y.Z: ..."
# 4. Tag + push
git tag -a vX.Y.Z -m "vX.Y.Z: short summary"
git push origin master --tags
```

The `release.yml` workflow then auto-creates the GitHub Release with `install.sh` + tarball + sha256 sidecar. If `HOMEBREW_TAP_TOKEN` is set, the homebrew formula is also bumped automatically.

Manual homebrew bump (if auto-bump unavailable)

```
SHA=$(curl -fsSL https://github.com/preston-check/preston-
        check/releases/download/vX.Y.Z/preston-check-X.Y.Z.tar.gz.sha256 | awk '{print
        $1}')
git -C /tmp/homebrew-tap pull
```

```
# Edit Formula/preston-check.rb: url, sha256, version
git -C /tmp/homebrew-tap commit -am "preston-check X.Y.Z"
git -C /tmp/homebrew-tap push
```

One-off Cloudflare admin tasks

```
export CLOUDFLARE_API_TOKEN="..."   # from repo secrets, or operator's local
export CLOUDFLARE_ACCOUNT_ID="..."

# List all Pages projects
curl -s -H "Authorization: Bearer $CLOUDFLARE_API_TOKEN" \
  "https://api.cloudflare.com/client/v4/accounts/$CLOUDFLARE_ACCOUNT_ID/pages/projects" \
  | python3 -m json.tool

# Query the telemetry D1 directly
wrangler d1 execute preston-check-telemetry --remote --command "SELECT count(*) FROM
        scans"

# Tail the customer-portal Function logs
wrangler pages deployment tail --project-name=preston-check-customer
```

Emergency response

Compromised Cloudflare API token

1. `https://dash.cloudflare.com/profile/api-tokens` → click "…" next to the token → **Revoke**
2. Create a new token with the same scopes (Cloudflare Pages: Edit, Workers Scripts: Edit, Account Settings: Read, Workers KV Storage: Edit, D1: Edit)
3. `gh secret set CLOUDFLARE_API_TOKEN --repo preston-check/preston-check` → paste new value
4. Verify next deploy still succeeds (push a no-op or run `gh workflow run`)

Compromised license signing key

The Ed25519 keypair lives at `~/.preston-check/keys/private.pem` on the operator's hardware-bound machine. If compromised:

1. Generate a new keypair via `tools/setup-signing-key.sh`
2. Replace `lib/license_pubkey.pem` with the new public half
3. Re-issue every active license against the new key
4. Push a new release (any vX.Y.Z) so customers get the updated public key
5. The old private key file gets deleted; old licenses no longer verify after the next pull

Customer portal compromised / data leak

Customer portal currently has no real customer data — mock skeleton only. If real data has been migrated and a leak is suspected:

1. Cloudflare dashboard → Workers & Pages → preston-check-customer → temporarily delete or pause the deployment
2. Investigate logs via Cloudflare Pages → Functions → Logs
3. Rotate any database credentials referenced in the project bindings
4. Issue customer notifications via the published security disclosure policy in `SECURITY.md`

Lost operator access to admin portal

Cloudflare Access is the gate. If the operator's email is no longer accessible:

1. Cloudflare Zero Trust dashboard → Access → Applications → Preston-Check Admin → edit the policy
2. Change the Allow rule's email to the new operator address
3. Save — next visit to `admin.preston-check.com` prompts for the new email

If the entire Zero Trust account is locked:

1. Use the backup recovery contact email if one is configured
2. If not, follow Cloudflare's account recovery flow at `https://dash.cloudflare.com/sign-in`
3. The deploy infrastructure continues working without admin portal access (admin is only needed for operator UI; deploys go through GitHub Actions independently)

Monitoring & observability

What's currently in place:

- **Cloudflare Pages dashboard** — deploy history, function logs, custom domain status. Per-project at `dash.cloudflare.com/<account>/pages/view/<project>`.
- **GitHub Actions tab** — every workflow run, with logs. `https://github.com/preston-check/preston-check/actions`
- **D1 query interface** — `wrangler d1 execute preston-check-telemetry --remote --command "..."` for ad-hoc queries against the telemetry dataset.
- **Cloudflare Web Analytics** — not yet enabled; the bound projects' "Web analytics" tab in the Pages dashboard activates it.

What's not in place yet:

- Sentry / Honeycomb / Datadog integration — the eventual goal but the project is too low-volume to justify yet
- Status page — public uptime monitor at `https://status.preston-check.com/`

- Daily KPI digest email — pulls from D1 + the GitHub stats API, lands in operator inbox

Anonymity / pseudonymity notes

The current Cloudflare account is registered to the operator's pre-pseudonym email. Long-term hygiene per `docs/strategy/anonymity-and-mystique.md`:

1. Form a Wyoming LLC ("Catalog Holdings LLC" or similar)
2. Register a fresh Cloudflare account using `operator@<llc-domain>` or `preston@preston-check.com`
3. Migrate Pages projects + Worker + D1 + KV to the new account
4. Update `CLOUDFLARE_ACCOUNT_ID` repo secret with the new account's ID
5. Delete the old account once everything is verified on the new one

Until then: the workers.dev subdomain has been renamed from a personal one to `preston-check-edge.workers.dev` (anonymity-clean), and operator-facing emails are role-based (`hello@`, `security@`, `support@`, `press@`, `preston@` all forward to the operator's actual mailbox via Cloudflare Email Routing or equivalent).

Where things live (file-system map)

```
/web/landing/    — public landing page (deploys to preston-check.com via GitHub Pages)
/web/landing/icons/    — proprietary 23-icon SVG set, shared by all surfaces
/web/landing/assets/    — logomark, wordmark, score-badge, watermark
/web/admin/      — admin portal SPA (deploys to admin.preston-check.com via Cloudflare
Pages)
/web/customer/   — customer portal SPA (deploys to app.preston-check.com via Cloudflare
Pages)
/web/customer/functions/api/v1/telemetry/index.ts   — telemetry endpoint (Pages
Function)
/workers/telemetry/   — standalone telemetry Worker (alternative to Pages Function)

/lib/   — runner libraries (license, telemetry, ai_analyze, ai_autofix, branding,
oss_detection)
/checks/   — catalog (294 checks total: core + community/{verified,accepted,proposed})
/modules/smart-contract-audit/   — separate runner for deep contract audits
/tools/   — operator scripts (issue-license, setup-signing-key, sync-threat-intel,
integrations)
/.github/workflows/   — eight CI workflows (pages, admin-pages, customer-pages,
telemetry-deploy, release, test, lint-community, threat-intel-sync)
/docs/   — design docs (architecture, portals-and-kpis, strategy/, etc.)
```

Updating this runbook

This file lives at `docs/operator-runbook.md` in the public repo. Keep it current — every time a new resource is provisioned, a URL changes, or an emergency procedure needs adjustment, update this file in the same commit. The file is intentionally short on prose and long on tables so it stays scannable at 3am during an incident.